Präsentation

Spickzettel: Ablauf, Demo-Drehbuch, Architektur, Begriffe und 30 Rückfragen

1 Ablauf (15 Minuten)

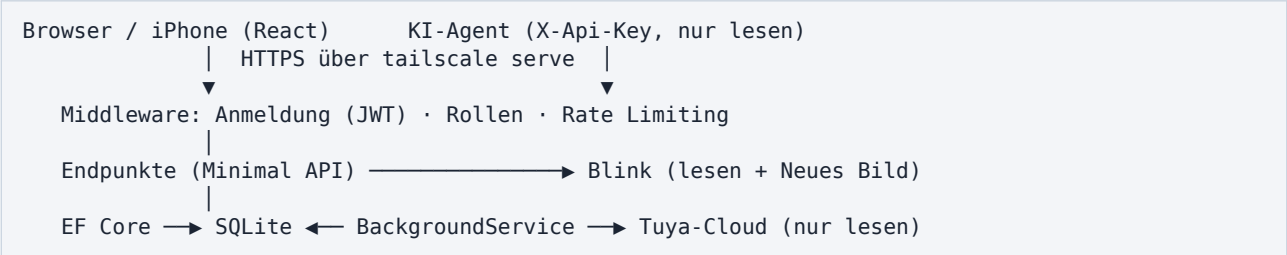
Zeit	Teil	Kernsatz
0:00	Einstieg	Mit einem Blick sehen, was zu Hause los ist, ohne fünf Apps.
1:00	Ziel	Liest Tuya und Blink, schaltet bewusst nichts.
2:30	Architektur	React → Minimal API → EF Core → SQLite, dazu ein Hintergrunddienst.
4:30	Live-Demo	Echter Code, nur die Cloud dahinter ist gespielt.
10:00	Technik	Schwierig waren Anmeldung, Geheimnisse und Kontingent.
12:00	Grenzen, Gelernt	Polling verpasst kurze Türöffnungen – ich habe einen Plan.
13:30	Ausblick, Fragen	Push über Pulsar, MCP-Server für KI-Assistenten.

Vorher: ./demo-starten.sh · localhost:5300 · demo@example.de · Zoom 125 % · Handy-Tab 390 px · Plan B: docs/images/screenshots/. Zu knapp? Reports und Einstellungen weglassen.

2 Demo-Drehbuch

#	Klick	Sagen
1	Anmeldeseite (Banner)	Zweite Instanz mit Beispielhaus, keine echten Daten.
2	Übersicht	Ein Satz sagt, ob alles in Ordnung ist.
3	Status-Leiste	Drucker offline. LEDs lesen den Speicher, kosten keine Tuya-Aufrufe.
4	Küche: Kühlschrank	Tuya liefert ganze Zahlen, Watt wird durch 10 geteilt.
5	Energie, Batterien	Ab 20 % gelb: Haustür 15 %.
6	Geräte	Verknüpfen mit Tuya, danach gleicht der Hintergrunddienst ab.
7	Haustür → Ereignisse	Jede Änderung ist ein Ereignis, daraus entstehen Reports.
8	Mit Tuya abgleichen	Sofort von Hand, nur lesen.
9	Reports	Ein UTC-Tag, so wie die Zeiten gespeichert sind.
10	Blink	Gelb = älter als 1 Tag. Bilder über den Server, kein Token im Browser.
11	Neues Bild	Einzige Ausnahme: nur Admin, 1× pro Minute, kostet Akku.
12	Nochmal Neues Bild	429: Rate Limit greift.
13	Einstellungen	Nur Admin. Tür alle 3 min, Rest alle 30 min (Kontingent).
14	Handy 390 px	PWA auf dem iPhone, HTTPS über Tailscale.

3 Architektur



Demo: Nur die Tuya-Cloud (HTTP-Ebene) und die Kamera (Interface) sind ersetzt, per Dependency Injection.

4 Begriffe in einem Satz

Begriff	Satz
Minimal API	Endpunkte als Funktionen mit app.MapGet, ohne Controller.
DTO	Eigene Klasse für Ein- und Ausgabe, das Datenbankmodell bleibt innen.
EF Core / Migration	ORM mit parametrisiertem SQL; Schemaänderung als Version, alte nie ändern.
Dependency Injection	Das Framework erzeugt Dienste und reicht sie weiter.
Middleware	Kette für jede Anfrage, Reihenfolge zählt (Auth vor Rate Limit).
JWT	Signiertes Token mit Benutzer und Rolle, 15 Minuten gültig.
Refresh-Token	HTTP-only-Cookie, 30 Tage, bei jedem Gebrauch ersetzt (Rotation).
Reuse Detection	Altes Refresh-Token wieder benutzt → alle Sitzungen enden.
Identity	Benutzer, Rollen, Passwort-Hash (PBKDF2), Sperre nach 5 Fehlversuchen.
Rate Limiting	Zu viele Anfragen im Zeitfenster → 429 mit Retry-After.
BackgroundService	Läuft neben der API dauerhaft, hier der Tuya-Abgleich.
Polling / Push	Selbst regelmäßig fragen / die Gegenseite meldet sich.
HMAC	Signatur jeder Tuya-Anfrage mit dem geheimen Schlüssel.
User Secrets	Geheimnisse im Benutzerordner, nie im Repo.
Environment	Development, Production, Demo: andere Einstellungen und Schutz.
Fake	Ersatz für eine echte Abhängigkeit, Tests und Demo ohne Internet.
PWA	Web-App auf dem Home-Bildschirm.
Reverse Proxy	tailscale serve nimmt HTTPS an und reicht nach innen weiter.
Image / Container	Bauplan / laufende Kopie davon.

5 30 Rückfragen – Kurzantworten

Frage	Antwort
1 Warum nichts schalten?	Lesen ist ungefährlich, Fokus Überblick.
2 Warum Neues Bild?	Verändert nichts im Haus. Admin, 1×/min, Akku.
3 Minimal API?	Weniger Code, im Unterricht, übersichtlich.
4 SQLite?	Eine Datei, reicht fürs Zuhause, Wechsel mit EF Core leicht.
5 SQL-Injection?	EF Core parametrisiert, nie SQL aus Text.
6 Anmeldung?	Access Token 15 min + Refresh-Cookie 30 Tage, Rotation.
7 Warum zwei Tokens?	Kurz = wenig Schaden; Cookie für JavaScript unlesbar.
8 Gestohlenes Refresh?	Reuse Detection beendet alle Sitzungen.
9 Passwörter?	Nur Hash (PBKDF2) über Identity.
10 Passwort-Raten?	10 Versuche/min pro Gerät, Sperre nach 5 Fehlern.
11 Tuya-Zugang?	User Secrets bzw. .env, nie im Repo.
12 Wie oft Tuya?	Standard 30 min, Tür 3 min, ca. 20 500 Aufrufe/Monat.
13 Warum nicht öfter?	Kontingent begrenzt, Takt gerechnet.
14 Tuya fällt aus?	Backoff bis 1 h, LED rot, App läuft weiter.
15 Verpasste Tür?	Polling sieht nur Abfragezeitpunkte → Push.
16 Handy?	Tailscale (VPN), tailscale serve mit HTTPS.
17 Aus dem Internet?	Nein, nur Tailnet, Backend auf 127.0.0.1.
18 Blink?	Inoffizielle API, eigener Client (Blink.NET: 426).
19 Bilder über Server?	Kein Token im Browser, kein Cache.
20 Demo technisch?	Fakes per Dependency Injection, sonst gleicher Code.
21 Demo im Betrieb?	Startet nicht; eigene Datenbank mit „demo“ im Namen.
22 Tests?	Über 230 xUnit-Tests, API im Arbeitsspeicher, Fakes.
23 Fake-Uhr?	TimeProvider: Zeit vorspulen statt warten.
24 pruefen.sh?	Build, Migrationen, Tests, Frontend, Lint – vor jedem Commit.
25 Wer darf was?	Admin schreibt, Nur lesen liest, Agent liest mit Schlüssel.
26 Kein UI-Framework?	Eigenes CSS mit Tokens: klein, verstanden.
27 Docker?	Läuft überall gleich, nicht als root, Healthcheck.
28 Rolle der KI?	Werkzeug; Entscheidung, Prüfung, Verantwortung bei mir.
29 Schwierigstes?	Refresh-Rotation und Blink (426).
30 Heute anders?	Früher CI und sicherer Leseschutz als Standard.

6 Gelernt und Ausblick

Gelernt: Ein Projekt hängt von Datenbank bis Betrieb zusammen · Sicherheit sind viele kleine Stellen · Fremde APIs planen (Kontingent, Ausfall) · Tests mit Fakes machen mutig · Branch, PR, Prüfskript · Mit KI arbeiten, ohne das Denken abzugeben.
Ausblick: Push über Tuya Message Service (Pulsar) · MCP-Server für KI-Assistenten · Server mit Docker · Blink-Clips · Audit-Log · CI.